

Three.js 実装変更まとめ

このファイルは、`main.js` に追加・変更した内容を説明したものです。主な変更は次の 3 つです。

- `images/` の素材を使った本モデルの追加
- 表紙の `ura.jpg` + `プラネテス透過.png` 重ね合わせと、金色部分の光沢表現
- FPS 移動中に GLB オブジェクトへぶつかる当たり判定の追加

1. 本モデルの追加

対象箇所: `main.js` 77 行目付近から

`createBook()` という関数を追加し、Three.js の基本形状を組み合わせて本を作っています。

```
function createBook() {  
  const book = new THREE.Group();  
  
  const coverWidth = 1.35;  
  const coverHeight = coverWidth * (790 / 569);  
  const bookDepth = coverHeight * (153 / 790);  
  const boardDepth = 0.045;  
  const coverOverhang = 0.035;  
  const spineWidth = 0.095;
```

各値の意味

- `coverWidth`: 表紙の横幅
- `coverHeight`: 表紙の高さ。画像比率 `569 x 790` に合わせて計算
- `bookDepth`: 本の厚み。背表紙画像の比率 `153 x 790` に合わせて計算
- `boardDepth`: 表紙・裏表紙の板の厚み
- `coverOverhang`: 表紙が本文ページより少し大きく見えるようにする余白
- `spineWidth`: 背表紙部分の横幅

最初は単純な箱に画像を貼る形でしたが、リアルさを出すために現在は以下のパーツに分けています。

- `pageBlock`: 本文ページのかたまり
- `frontCover`: 表紙の厚み付き板
- `frontGold`: 表紙の金色デザイン用オーバーレイ
- `backCover`: 裏表紙の厚み付き板
- `spine`: 背表紙
- `pageLines`: 小口側のページ線

2. テクスチャ読み込み処理

対象箇所: `main.js` 80 行目付近

画像読み込み用に `loadBookTexture()` を追加しました。

```
function loadBookTexture(path) {  
  const texture = textureLoader.load(path);  
  texture.colorSpace = THREE.SRGBColorSpace;  
  texture.wrapS = THREE.ClampToEdgeWrapping;  
  texture.wrapT = THREE.ClampToEdgeWrapping;  
  texture.anisotropy = renderer.capabilities.getMaxAnisotropy();  
  return texture;  
}
```

ここでやっていること

- `THREE.SRGBColorSpace` を指定して、画像の色がくすみにくいようにする
- `ClampToEdgeWrapping` で画像端の繰り返しを防ぐ
- `anisotropy` を最大にして、斜めから見たときのテクスチャのぼやけを軽減する

3. 表紙画像の重ね合わせ

対象箇所: `main.js` 105 行目付近

表紙は `ura.jpg` をベースにし、その上に `プラネテス透過.png` を重ねています。

```
const frontTexture = loadBookTexture("./images/ura.jpg");  
const frontGoldTexture = loadBookTexture("./images/プラネテス透過.png");  
const backTexture = loadBookTexture("./images/ura.jpg");  
const spineTexture = loadBookTexture("./images/sebyoushi.jpg");
```

表紙ベース

`frontMaterial` は `ura.jpg` をそのまま表紙面に貼るためのマテリアルです。

```
const frontMaterial = new THREE.MeshStandardMaterial({  
  map: frontTexture,  
  roughness: 0.7,  
  metalness: 0  
});
```

金色オーバーレイ

`frontGoldMaterial` は、透過 PNG の金色部分を少しメタリックに見せるためのマテリアルです。

```
const frontGoldMaterial = new THREE.MeshPhysicalMaterial({  
  map: frontGoldTexture,  
  color: 0xffedb0,  
  metalness: 0.35,  
  roughness: 0.22,
```

```
clearcoat: 0.8,  
clearcoatRoughness: 0.08,  
emissive: 0x8a5a18,  
emissiveMap: frontGoldTexture,  
emissiveIntensity: 0.35,  
transparent: true,  
alphaTest: 0.08,  
depthWrite: false,  
side: THREE.FrontSide  
});
```

金色表現の調整ポイント

- **metalness**: 大きくすると金属感が増える。ただし強すぎると暗く見えやすい
- **roughness**: 小さくすると光沢が強くなる
- **clearcoat**: 表面のニスのようなツヤ
- **emissiveIntensity**: 金色が暗く沈みすぎないように少し発光させる
- **alphaTest**: 透過 PNG の透明部分を切り抜くしきい値
- **depthWrite: false**: 透過オーバーレイが表紙面と描画競合しにくくする

表紙の上にオーバーレイを置く部分はここです。

```
const frontGold = new THREE.Mesh(  
  new THREE.PlaneGeometry(coverWidth + coverOverhang, coverHeight +  
  coverOverhang * 2),  
  frontGoldMaterial  
);  
frontGold.position.z = frontSurfaceZ;  
frontGold.castShadow = false;  
frontGold.receiveShadow = true;  
book.add(frontGold);
```

frontSurfaceZ は表紙面より少し手前の位置です。

```
const frontSurfaceZ = bookDepth / 2 + boardDepth + 0.004;
```

この **0.004** のずらしで、表紙面と金色オーバーレイが同じ平面に重なってチラつく現象を防いでいます。

4. 角欠け・背表紙のガタつき対策

以前は表紙や背表紙の面がほぼ同じ位置に重なり、カメラ角度によって描画がチラつく状態がありました。

対策として、背表紙の奥行きを少しだけ小さくしています。

```
const spineDepth = bookDepth + boardDepth * 2 - 0.018;
```

背表紙メッシュはこの `spineDepth` を使っています。

```
const spine = new THREE.Mesh(  
  new THREE.BoxGeometry(spineWidth, coverHeight + coverOverhang * 2,  
    spineDepth),  
  [  
    coverEdgeMaterial,  
    spineMaterial,  
    coverEdgeMaterial,  
    coverEdgeMaterial,  
    coverEdgeMaterial,  
    coverEdgeMaterial  
  ]  
);
```

これにより、表紙・裏表紙の前後面と背表紙の前後面が同じ平面で重ならなくなり、z-fighting によるガタつきを抑えています。

5. ページ側面の表現

対象箇所: `main.js` 227 行目付近

小口側に細い線を複数入れて、ページが重なっているように見せています。

```
const pageLines = new THREE.Group();  
const lineCount = 28;  
for (let i = 0; i < lineCount; i++) {  
  const y = -coverHeight * 0.43 + (coverHeight * 0.86 * i) / (lineCount -  
    1);  
  const points = [  
    new THREE.Vector3(pageSideX, y, -bookDepth / 2 + 0.012),  
    new THREE.Vector3(pageSideX, y + Math.sin(i * 1.7) * 0.004, bookDepth /  
    2 - 0.012)  
  ];  
  const geometry = new THREE.BufferGeometry().setFromPoints(points);  
  pageLines.add(new THREE.Line(geometry, pageLineMaterial));  
}  
book.add(pageLines);
```

なぜ背表紙側には線を出していないか

背表紙付近に細かい線を置くと、近距離でガタガタした見た目になりやすかったためです。現在は小口側だけに線を置いて、本らしさを残しつつ背表紙側を安定させています。

6. 本用のハイライトライト

対象箇所: `main.js` 361 行目付近

金色部分の光沢が見えやすくなるように、表紙方向へ暖色のポイントライトを追加しています。

```
const bookHighlight = new THREE.PointLight(0xffe1a3, 1.2, 5);
bookHighlight.position.set(0.2, 2.3, -1.4);
scene.add(bookHighlight);
```

調整ポイント

- 第 1 引数 `0xffe1a3`: 光の色。暖色寄り
- 第 2 引数 `1.2`: 光の強さ
- 第 3 引数 `5`: 光が届く距離
- `position`: 金色表紙にハイライトが入りやすい位置

7. GLB オブジェクトの当たり判定

対象箇所: `main.js` 254 行目付近

GLB 用の当たり判定リストを追加しました。

```
const glbColliders = [];  
const playerRadius = 0.35;  
const playerHeight = 1.8;
```

各値の意味

- `glbColliders`: GLB の当たり判定ボックスを保存する配列
- `playerRadius`: プレイヤーの横幅。大きくすると早めにぶつかる
- `playerHeight`: プレイヤーの高さ。カメラ位置から下方向へこの高さぶんを判定する

GLB の読み込み後、`THREE.Box3` でモデル全体を囲むボックスを作っています。

```
function addGLBCollider(model, name) {  
  model.updateMatrixWorld(true);  
  const box = new THREE.Box3().setFromObject(model);  
  glbColliders.push({ name, box });  
  console.log(`${name} の当たり判定を追加しました。`, box);  
}
```

各 GLB の `scene.add()` の直後に、当たり判定登録を追加しています。

```
scene.add(model1);  
addGLBCollider(model1, "モデル1");
```

```
scene.add(model2);
```

```
addGLBCollider(model2, "モデル2");
```

```
scene.add(model3);  
addGLBCollider(model3, "モデル3");
```

この順番が重要です。位置・回転・スケールを設定した後に `addGLBCollider()` を呼ぶことで、実際に表示されている位置に合わせた当たり判定になります。

8. FPS 移動と衝突チェック

対象箇所: `main.js` 389 行目付近

まず、円形ステージ内にいるかを判定します。

```
function isInsideGround(position) {  
  playerXZ.set(position.x, 0, position.z);  
  return playerXZ.distanceTo(groundCenter) <= groundRadius - playerRadius;  
}
```

次に、プレイヤーが GLB の当たり判定ボックスにぶつかっているかを判定します。

```
function isPlayerCollidingWithGLB(position) {  
  const playerBottom = position.y - playerHeight;  
  const playerTop = position.y;  
  
  for (const collider of glbColliders) {  
    const box = collider.box;  
  
    if (playerTop < box.min.y || playerBottom > box.max.y) {  
      continue;  
    }  
  
    const closestX = THREE.MathUtils.clamp(position.x, box.min.x,  
      box.max.x);  
    const closestZ = THREE.MathUtils.clamp(position.z, box.min.z,  
      box.max.z);  
    const dx = position.x - closestX;  
    const dz = position.z - closestZ;  
  
    if (dx * dx + dz * dz < playerRadius * playerRadius) {  
      return true;  
    }  
  }  
  
  return false;  
}
```

判定の考え方

- プレイヤーは「半径 `playerRadius` の円柱」として扱う
- GLB は `Box3` の直方体として扱う
- 高さが重なっていない場合は衝突しない
- XZ 平面上で、プレイヤー円が GLB の箱に触れたら衝突

ステージ内で、かつ GLB にぶつかっていないければ移動可能です。

```
function canPlayerStandAt(position) {  
  return isInsideGround(position) && !isPlayerCollidingWithGLB(position);  
}
```

9. 移動処理の変更

以前の移動処理は、押されたキーに応じてそのままカメラを動かしていました。

```
controls.moveRight(velocity.x);  
controls.moveForward(velocity.z);
```

現在は `movePlayerWithCollision()` に置き換えています。

```
movePlayerWithCollision(velocity.x, velocity.z);
```

実装は次の通りです。

```
function movePlayerWithCollision(rightAmount, forwardAmount) {  
  const player = controls.getObject();  
  const beforeMove = player.position.clone();  
  
  if (rightAmount !== 0) {  
    controls.moveRight(rightAmount);  
  
    if (!canPlayerStandAt(player.position)) {  
      player.position.copy(beforeMove);  
    } else {  
      beforeMove.copy(player.position);  
    }  
  }  
  
  if (forwardAmount !== 0) {  
    controls.moveForward(forwardAmount);  
  
    if (!canPlayerStandAt(player.position)) {  
      player.position.copy(beforeMove);  
    }  
  }  
}
```

```
}  
}  
}
```

なぜ横移動と前後移動を分けているか

横移動と前後移動を同時に判定すると、斜めに壁へ当たったとき完全に止まりやすくなります。

現在は、

1. 横方向に動けるか確認
2. 動けたらその位置を保存
3. 前後方向に動けるか確認
4. ぶつかった方向だけ戻す

という流れにしています。これにより、GLB の横をすべるような FPS らしい移動になります。

10. 動作確認

確認した内容:

- `node --check main.js` で構文エラーなし
- ローカルサーバー `http://localhost:8000/` で素材配信を確認
- GLB ファイルと表紙素材が HTTP 200 で読み込まれていることを確認

11. 今後調整しやすい値

プレイヤーの当たり判定

```
const playerRadius = 0.35;  
const playerHeight = 1.8;
```

- GLB に近づきすぎる場合: `playerRadius` を大きくする
- GLB から離れすぎる場合: `playerRadius` を小さくする
- 高さ方向の判定を変えたい場合: `playerHeight` を調整する

移動速度

```
velocity.copy(direction).multiplyScalar(0.04);
```

- 速くしたい場合: `0.04` を大きくする
- 遅くしたい場合: `0.04` を小さくする

本の大きさ


```
const coverWidth = 1.35;
```

本全体のサイズはこの値を基準にしているため、ここを変えると表紙・裏表紙・背表紙・ページ部分のサイズも連動して変わります。